

MANUAL CSV TRUTH-VALIDATION SOP

1. PURPOSE OF THIS TASK

Your job is to **manually verify whether the products returned for each search query are actually correct and relevant to that query.**

You will receive:

1. A **JSON file** containing search queries and their retrieved products.
2. A **master** `product_catalog.json` containing the authoritative product catalog.
3. Your task is to manually inspect the JSON results and create a CSV containing the required columns.

STRICT RULE

This is a **manual human-validation task.**

You **MUST NOT** use:

- AI
- ChatGPT
- Other LLMs
- GitHub Copilot
- Copilot Agent
- AI coding assistants
- AI browser assistants
- Automatic classification tools
- Scripts that decide whether a result is correct
- Any other automated system to determine correctness

You may use normal file viewing, text search, spreadsheet software, a text editor, and basic tools required to inspect the provided JSON/catalog.

The final judgment must be made by you.

2. INPUT JSON STRUCTURE

The JSON provided to you may contain fields similar to:

```
{
  "query": [...],
  "modified_query": [...],
```

```
"category": [...],
"inferred_section": [...],
"is_b2b": [...],
"is_exact_match": [...],
"total_hits": [...],
"brand": [...],
"sku_name": [...]
}
```

Each row/index represents one query and its corresponding search output.

The most important fields for this task are:

- query
- sku_name

Other fields can provide context but **do not replace your own manual judgment**.

3. CSV OUTPUT FORMAT — EXACT COLUMN NAMES

Your final CSV **MUST contain these columns with exactly these names and spelling**:

```
Query,Output,Correct,Comments,Improved
```

Do **not**:

- rename columns
- change capitalization
- add spaces to column names
- remove columns
- add extra columns
- reorder the columns

The expected order is:

Query	Output	Correct	Comments	Improved
-------	--------	---------	----------	----------

4. MEANING OF EACH COLUMN

4.1 Query

Copy the original query exactly as provided.

Do not rewrite it.

Do not improve grammar.

Do not normalize spelling unless the source JSON already contains that version.

Example:

```
Query
refrigerator
```

4.2 Output

Write the products returned in the JSON `sku_name` field.

IMPORTANT: `sku_name` **IS AN ARRAY**

For example:

```
"sku_name": [
  "LG 260L Double Door Refrigerator",
  "Samsung 253L Frost Free Refrigerator",
  "Whirlpool 265L Refrigerator"
]
```

In the CSV, the `Output` cell should contain the relevant product names.

Example:

```
LG 260L Double Door Refrigerator; Samsung 253L Frost Free Refrigerator;
Whirlpool 265L Refrigerator
```

Use a **consistent separator**, preferably:

;

between multiple products.

Simple generic queries

For broad queries such as:

refrigerator

or:

four wheeler

the output may legitimately contain multiple products of the same general type.

Therefore, you should **not mark results wrong merely because multiple refrigerators or multiple cars are returned.**

The real question is whether the returned products satisfy the complete intent of the query.

5. THE MOST IMPORTANT RULE — MATCH THE ENTIRE QUERY INTENT

A result is **NOT automatically correct merely because it belongs to the same product category.**

You must evaluate **ALL meaningful requirements contained in the query.**

These requirements can include:

- Product type
- Brand
- Price
- Minimum price
- Maximum price
- Budget
- Under/over a certain price
- Family requirement
- Capacity
- Size

- Storage
- RAM
- Processor
- Screen size
- Fuel type
- Seating capacity
- Number of doors
- Compatibility
- Model specification
- Usage purpose
- Professional/B2B requirement
- Other explicit attributes
- Other concepts expressed naturally in the query

CORE PRINCIPLE

The query must be evaluated as a complete requirement, not as a collection of individual words.

6. EXAMPLE OF A CORRECT GENERAL CATEGORY MATCH

Query

refrigerator

Returned output:

LG 260L Refrigerator
Samsung 253L Refrigerator
Whirlpool 265L Refrigerator

This can be marked:

Correct = 1

because the query is broad and the products are refrigerators.

You do **not** need an exact single product for a broad query.

7. EXAMPLE OF AN INCORRECT ATTRIBUTE MATCH

Query

laptop with 16GB RAM

Returned output:

HP Laptop 8GB RAM
Dell Laptop 8GB RAM
Lenovo Laptop 4GB RAM

These are laptops, but they do **not** satisfy the important query requirement of:

16GB RAM

Therefore:

Correct = 0

The fact that they are laptops is **not enough**.

8. EXAMPLE — STORAGE REQUIREMENT

Query

phone with 256GB storage

Returned:

Samsung Phone 128GB
Apple Phone 128GB
OnePlus Phone 64GB

This is:

Correct = 0

because the products do not satisfy the requested storage requirement.

9. EXAMPLE — BUDGET REQUIREMENT

Query

laptop under 50000

Returned:

Laptop A - ₹42,000
Laptop B - ₹47,000
Laptop C - ₹39,000

These satisfy the budget.

Therefore they can be:

Correct = 1

But suppose the output is:

Laptop A - ₹42,000
Laptop B - ₹57,000
Laptop C - ₹72,000

Then the result does not satisfy the query as a complete set of retrieved products because products above the specified budget have been returned.

Therefore:

Correct = 0

IMPORTANT

Words such as:

- under
- below
- less than
- within
- budget
- affordable
- above
- over
- more than
- between

are meaningful constraints.

Do not ignore them.

10. EXAMPLE — PRICE RANGE

Query

cars between 8 lakh and 12 lakh

A valid result should respect that price range.

A result containing only cars priced at:

₹4 lakh
₹5 lakh
₹20 lakh
₹25 lakh

is not a valid match merely because they are cars.

Mark:

Correct = 0

11. EXAMPLE — FAMILY / USE-CASE REQUIREMENT

Queries may contain concepts rather than precise numeric specifications.

Examples:

refrigerator for family

car for a large family

family car under 10 lakh

phone suitable for gaming

laptop for office work

These are **not meaningless words**.

You must consider the intended use or concept when evaluating relevance.

For example, if a query asks for a family-oriented vehicle and the output consists of products that clearly do not satisfy the implied requirement, do not mark it correct simply because they are all cars.

Use the available information in the catalog to determine relevance.

12. APPLY ATTRIBUTE LOGIC ACROSS ALL PRODUCT CATEGORIES

The same validation logic applies regardless of product category.

For example:

Phones

Check:

- RAM

- Storage
- Price
- Brand
- Camera
- Battery
- Processor
- Other explicit requirements

Laptops

Check:

- RAM
- Storage
- Processor
- Screen
- Price
- Brand
- GPU
- Intended usage
- Other explicit requirements

Cars

Check:

- Price
- Seating
- Fuel
- Body type
- Family/use case
- Brand
- Transmission
- Other stated requirements

Appliances

Check:

- Capacity
- Energy rating
- Type
- Price
- Family/use case
- Brand
- Other explicit requirements

Any other category

Apply exactly the same principle:

Every meaningful requirement expressed by the user must be respected.

13. WHEN TO MARK Correct = 1

Set:

Correct = 1

ONLY when the returned output is genuinely relevant to the query as a whole.

The returned products should satisfy the important constraints expressed in the query.

Examples where Correct = 1 may apply:

Query

refrigerator

Output contains refrigerators.

Query

256GB phone

Output contains appropriate 256GB phones.

Query

laptop under 60000 with 16GB RAM

Output products satisfy both:

- the ₹60,000 budget
- 16GB RAM requirement

Query

Samsung refrigerator

Output contains appropriate Samsung refrigerators.

14. WHEN TO MARK Correct = 0

Set:

Correct = 0

when the output fails an important part of the query.

Typical reasons include:

- Wrong product category
- Wrong brand
- Wrong price range
- Wrong budget
- Missing required RAM
- Missing required storage
- Wrong size
- Wrong capacity
- Wrong usage concept
- Wrong specification
- Only partial matching
- Products that are related but do not satisfy the actual requirement
- A significant number of returned products violating the query constraints

IMPORTANT

Do **NOT** mark something correct simply because:

"It is somewhat related."

The standard is:

Does it genuinely satisfy the user's search intent?

15. **Comments** COLUMN RULE

The **Comments** column is primarily for **wrong results**.

When:

Correct = 0

you **MUST explain why**.

The explanation should be specific.

BAD COMMENT

Wrong

BAD COMMENT

Not relevant

GOOD COMMENT

Query requires 16GB RAM, but returned laptops have 8GB RAM.

GOOD COMMENT

Query specifies products under ₹50,000, but returned products include items above the requested budget.

GOOD COMMENT

Query asks for Samsung refrigerators, but returned products include LG and Whirlpool models.

GOOD COMMENT

Query requires 256GB storage, but the returned phones are 128GB and 64GB models.

The comment should identify the **specific mismatch**.

16. Comments FOR CORRECT RESULTS

For:

Correct = 1

you may use:

Matches the query requirements.

or leave the field blank if your team lead has instructed that comments are only required for incorrect rows.

However, **wrong rows must always have an explanation**.

17. Improved COLUMN — ONLY FOR WRONG RESULTS

When:

Correct = 0

you must identify the **correct products from the master** `product_catalog.json`.

These products go into:

Improved

VERY IMPORTANT

Do **NOT** invent product names.

Do **NOT** make up products.

Do **NOT** use products that do not exist in the master catalog.

The master catalog is the authoritative source for corrected outputs.

18. HOW TO SELECT Improved PRODUCTS

Use `product_catalog.json`.

Search the catalog manually for products that actually satisfy the query.

You must respect the same requirements discussed above.

Example

Query:

```
laptop under 60000 with 16GB RAM
```

Do not simply choose any cheap laptops.

The improved products must satisfy:

```
Laptop  
+  
Under ₹60,000  
+  
16GB RAM
```

If additional requirements are present, those must also be followed.

19. MAXIMUM NUMBER OF IMPROVED PRODUCTS

For a wrong result:

```
Improved
```

must contain **NO MORE THAN 4 PRODUCTS**.

You may provide:

1
2
3

or

4

valid products.

Never provide more than 4.

20. DO NOT IGNORE QUERY ATTRIBUTES WHEN CREATING Improved

This is one of the most important parts of the task.

Suppose the query is:

phone under 30000 with 8GB RAM and 256GB storage

You must find products satisfying:

Phone
AND
<= ₹30,000
AND
8GB RAM
AND
256GB storage

You must NOT return:

Phone A - ₹20,000, 6GB RAM, 128GB
Phone B - ₹25,000, 4GB RAM, 128GB

just because they are cheaper phones.

They do not satisfy the complete query.

21. WHAT TO DO FOR BROAD QUERIES

For a query such as:

refrigerator

or:

four wheeler

there may be many valid products.

This is acceptable.

The query is broad, so several products of the same category may be valid.

For example:

refrigerator

may reasonably return:

LG Refrigerator
Samsung Refrigerator
Whirlpool Refrigerator
Godrej Refrigerator

Do not mark such an output wrong merely because the products are multiple items of the same type.

22. DO NOT EXPECT ONLY ONE PRODUCT

The task is **not**:

"Find the single perfect product."

The task is:

"Determine whether the retrieved products are appropriate for the query."

Multiple relevant products are allowed.

23. WRONG CATEGORY = WRONG

Example:

Query

refrigerator

Output:

washing machine
air conditioner
microwave

Mark:

Correct = 0

because the products do not correspond to the requested category.

24. PARTIAL MATCH = NOT NECESSARILY CORRECT

Example:

Query

Samsung phone under 30000 with 8GB RAM

Output:

Samsung 8GB Phone - ₹28,000
Samsung 6GB Phone - ₹25,000
Samsung 4GB Phone - ₹20,000

Even though all are Samsung phones under ₹30,000, the latter two do not meet the 8GB RAM requirement.

Therefore the returned set fails an important query constraint.

Mark:

Correct = 0

and explain the mismatch.

25. THE "ALL REQUIREMENTS" TEST

Before marking any row, ask yourself:

Question 1

What product/category is being requested?

Question 2

Is there a brand requirement?

Question 3

Is there a budget or price requirement?

Question 4

Is there a minimum or maximum specification?

Question 5

Is there a storage/RAM/capacity/size requirement?

Question 6

Is there a family, usage, or other conceptual requirement?

Question 7

Are there any other constraints stated in the query?

Question 8

Do the returned products satisfy these requirements?

If the answer to an important requirement is **no**, the row should generally be:

Correct = 0

26. MANUAL WORKFLOW

Follow this process for **EVERY ROW**.

STEP 1 — Open the JSON

Open the provided JSON file using a normal text editor or JSON viewer.

STEP 2 — Identify the query

Read the value from:

query

Copy it exactly into:

Query

STEP 3 — Inspect sku_name

Read the corresponding:

sku_name

array.

These are the products currently returned by the system.

STEP 4 — Understand the COMPLETE query

Do not look only at the product category.

Identify every important requirement in the query.

For example:

gaming laptop under 70000 with 16GB RAM and 512GB SSD

requirements are:

gaming
laptop
under 70000
16GB RAM
512GB SSD

STEP 5 — Compare output against the requirements

Check whether the retrieved products satisfy the complete query.

STEP 6 — Fill Output

Write the returned products from `sku_name`.

STEP 7 — Fill Correct

Use only:

1

or:

0

No other values.

Allowed:

1
0

Not allowed:

yes
no
true
false
correct
wrong
Y
N

STEP 8 — Fill **Comments**

If wrong, explain exactly what failed.

STEP 9 — Fill **Improved**

Only for wrong rows:

1. Open `product_catalog.json`.
 2. Find valid products.
 3. Apply all query constraints.
 4. Select no more than 4 products.
 5. Enter those product names in `Improved`.
-

27. EXAMPLE OF A COMPLETE ROW

Suppose:

Query

```
laptop under 50000 with 16GB RAM
```

JSON output

```
HP Laptop 8GB RAM ₹42000  
Dell Laptop 8GB RAM ₹45000  
Lenovo Laptop 4GB RAM ₹39000
```

This is wrong.

CSV row:

```
Query,Output,Correct,Comments,Improved  
"laptop under 50000 with 16GB RAM","HP Laptop 8GB RAM ₹42000; Dell Laptop 8GB  
RAM ₹45000; Lenovo Laptop 4GB RAM ₹39000",0,"Returned laptops are within budget  
but do not satisfy the required 16GB RAM specification.","ASUS VivoBook 16GB RAM  
₹46990; Acer Aspire 16GB RAM ₹48990"
```

The exact improved products must come from the master catalog.

28. ANOTHER EXAMPLE

Query

```
refrigerator
```

Output

```
LG 260L Refrigerator  
Samsung 253L Refrigerator  
Whirlpool 265L Refrigerator
```

Possible CSV row:

```
"refrigerator","LG 260L Refrigerator; Samsung 253L Refrigerator; Whirlpool 265L  
Refrigerator",1,"Matches the requested product category.",""
```

29. IMPROVED PRODUCTS MUST COME FROM THE MASTER CATALOG

The master catalog is the source of truth.

Therefore:

NEVER

Invent:

Samsung XYZ Refrigerator

unless that exact product exists in the catalog.

NEVER

Modify an existing product name to make it fit.

NEVER

Create fictional specifications.

NEVER

Guess that a product exists.

NEVER

Use external product knowledge to fabricate a corrected SKU.

Only use products that actually exist in `product_catalog.json`.

30. IMPORTANT RULE FOR PRODUCT NAMES

Copy product names accurately.

Do not unnecessarily:

- Shorten names

- Rename products
- Rewrite product titles
- Add imaginary specifications
- Remove identifying model information

Use the product name as it appears in the catalog.

31. CSV FORMATTING RULES

The CSV must be a valid CSV file.

Because product names may contain commas, quotation marks, or other punctuation, make sure the CSV correctly quotes fields when necessary.

Example:

```
Query,Output,Correct,Comments,Improved
"refrigerator","LG 260L Refrigerator; Samsung 253L Refrigerator",1,"Matches the
requested category.", ""
```

Do not create malformed rows.

Every row must have the same five columns.

32. ROW COUNT

The number in the filename must equal the **total number of data rows actually filled in the CSV**.

Do not count the header as a data row.

For example:

```
250 data rows
```

means the filename is:

```
Truth_EmployeeName_250.csv
```

NOT:

```
Truth_EmployeeName_251.csv
```

33. FILE NAMING FORMAT

Every employee's final file **MUST** follow:

```
Truth_{employee_name}_{total rows numbers filled in csv}.csv
```

Example:

```
Truth_Rahul_250.csv
```

Another example:

```
Truth_Samay_1000.csv
```

Replace:

```
{employee_name}
```

with your actual employee name.

Replace:

```
{total rows numbers filled in csv}
```

with the exact number of completed data rows.

34. FINAL FILE CHECK BEFORE SUBMISSION

Before submitting your CSV, manually verify:

COLUMN CHECK

The first row is exactly:

Query, Output, Correct, Comments, Improved

CORRECT COLUMN

Contains only:

1

or:

0

WRONG ROW CHECK

Every row where:

Correct = 0

has a useful explanation in:

Comments

IMPROVED CHECK

Every wrong row has appropriate corrected products wherever a valid correction can be identified from the master catalog.

PRODUCT COUNT CHECK

Improved contains no more than:

4 products

per row.

CATALOG CHECK

Every product listed in Improved actually exists in:

```
product_catalog.json
```

QUERY CHECK

Every `Query` exactly matches the source query.

OUTPUT CHECK

Every `Output` corresponds to the products actually returned by the provided JSON.

FILE NAME CHECK

The filename follows exactly:

```
Truth_{employee_name}_{row_count}.csv
```

35. COMMON MISTAKES — DO NOT MAKE THESE

Mistake 1

Seeing "laptop" in the query and "laptop" in the output and immediately marking:

```
Correct = 1
```

This is wrong when the query has additional requirements.

Mistake 2

Ignoring budget.

Example:

```
phone under 20000
```

and returning:

₹35000 phones

is not correct.

Mistake 3

Ignoring technical specifications.

Example:

16GB RAM

but returning:

8GB RAM

is not correct.

Mistake 4

Ignoring brand restrictions.

Example:

Samsung refrigerator

and returning:

LG refrigerator

is not a valid match.

Mistake 5

Writing explanations such as:

Wrong output

This does not explain the problem.

Write the actual mismatch.

Mistake 6

Inventing improved products.

This is strictly prohibited.

Mistake 7

Returning more than 4 improved products.

Maximum:

4

Mistake 8

Using AI to decide correctness.

This defeats the purpose of the task.

36. QUALITY STANDARD

The expected standard is **semantic correctness**, not superficial keyword matching.

You are evaluating whether the search result answers the user's request.

A product that merely shares one keyword with the query is **not automatically a correct result**.

Think in terms of:

```
QUERY INTENT
↓
ALL REQUIREMENTS
↓
RETURNED PRODUCTS
↓
DO THEY SATISFY THE REQUIREMENTS?
↓
YES → Correct = 1
NO  → Correct = 0
↓
Explain mismatch
↓
Find corrected products from master catalog
```

37. NO AI SLOP / NO HALLUCINATION

The output must be based on the files provided and your own manual inspection.

The following are unacceptable:

- AI-generated comments that have not been manually verified
- Fabricated products
- Fabricated specifications
- Incorrect product names
- Invented catalog entries
- Random guesses
- Copy-pasted generic explanations that do not describe the actual error
- Incorrect CSV structure
- Incorrect row counts
- False `Correct = 1` markings
- False `Correct = 0` markings

Every row must be independently checked.

38. QA AND ACCOUNTABILITY

All submitted files are subject to QA review.

QA may check:

- Random rows

- Entire batches
- Product validity
- Catalog consistency
- Query/output consistency
- Correctness labels
- Comments
- Improved products
- CSV structure
- Filename/row-count accuracy

STRICT QUALITY REQUIREMENT

Any **AI slop, hallucination, fabricated product, fabricated information, knowingly incorrect labeling, careless validation, or materially incorrect CSV submission** identified during QA may result in **strict disciplinary action under company policy**.

Employees are responsible for the accuracy of the CSVs they submit.

Do not submit a file until you have manually verified it.

39. GOLDEN RULE

Remember this rule throughout the entire task:

A result is correct only when it satisfies the actual intent and meaningful constraints of the query — not merely because it belongs to the same general category.

For incorrect results:

Explain the exact problem and provide up to 4 genuinely suitable products from the master `product_catalog.json`.

Everything must be manually verified.

40. REQUIRED FINAL DELIVERABLE

At the end of the task, submit exactly one CSV per employee in this format:

`Truth_{employee_name}_{total_rows}.csv`

with exactly these columns:

Query, Output, Correct, Comments, Improved

No additional columns.

No missing columns.

No invented products.

No AI-generated decisions.

No unsupported assumptions.

No more than 4 improved products per incorrect row.

Accuracy is the priority.